VRIJE UNIVERSITEIT
AMSTERDAM

RESEARCH PROJECT · VUSEC

Implementing Intel® Processor Trace in HyperDbg

Low-overhead, hypervisor-level control-flow tracing , from CPU packets to full execution reconstruction

Masoud Rahimi VUSec · Supervisor: Prof. Bos

Vrije Universiteit Amsterdam

Why hardware tracing, inside a hypervisor

Software tracing is expensive & visible

Breakpoints and DBI (Pin, DynamoRIO) can slow targets 10–100× and perturb timing， and are easy to detect.

Intel PT is done by the CPU

The processor emits a compressed control-flow trace with typically <5% overhead and zero code modification.

HyperDbg is the ideal host

Ring -1 (VT-x / EPT), OS-independent and stealthy, it already runs in VMX-root, where PT is configured.

It unlocks new capabilities

Coverage-guided fuzzing, execution reconstruction, malware RE and post-mortem debugging， from one primitive.

What is Intel® Processor Trace?

- A hardware feature on modern Intel CPUs (partial on Broadwell, full since Skylake).
- The CPU emits a stream of compressed packets describing control flow and not the full instruction stream.
- It records only what a decoder cannot infer statically: conditional-branch decisions and indirect / return targets.
- Decoder + the program binary → reconstruct the exact executed path, instruction by instruction.
- Runs per logical processor and writes to physical memory via ToPA.

< 5%

typical runtime overhead

Branch-only

encoding , direct jumps come from the binary

Continuous

full trace, not sampled snapshots

CAPABILITIES

Features & the options that matter

WHAT to trace , filtering

Process (CR3) one address space via IA32_RTIT_CR3_MATCH

IP ranges up to 4 windows (ADDR0–ADDR3) → module / function scope

Privilege ring 0 (OS) and / or ring 3 (User)

HOW to trace , controls

BranchEn emit control-flow (COFI) packets

Timing TSC / MTC / CYC for cycle-accurate traces

DisRETC emit explicit return targets (no RET compression)

ToPA direct output to physical memory

CONFIGURED THROUGH MSRs

`RTIT_CTL` `RTIT_STATUS` `RTIT_OUTPUT_BASE` `RTIT_OUTPUT_MASK_PTRS` `RTIT_CR3_MATCH` `RTIT_ADDRn_A / B`

Trace packets , the control-flow core

TNT *Taken / Not-Taken*

Conditional-branch outcomes, bit-packed (1 = taken, 0 = fell through). Up to 6 per short packet , the backbone of PT compression.

TIP *Target IP*

Targets of indirect branches, exceptions and interrupts. IP-compressed against the last IP to save bytes.

FUP *Flow Update*

The source IP bound to an asynchronous event (interrupt / exception) , tells the decoder where execution was.

TIP.PGE / PGD *Packet-gen enable / disable*

Mark exactly where tracing turns on and off (e.g. entering / leaving a filtered region).

Direct jumps and calls are never recorded, they are deterministic, so the decoder recovers them from the binary.

Context, synchronisation & timing

PIP
A CR3 change

MODE.Exec / TSX
Execution mode (16 / 32 / 64-bit) and transactional-memory status.

PSB / PSBEND
Periodic sync points; “PSB+” snapshots processor state so the decoder can resync.

OVF
Internal PT FIFO overflow , packets outran the drain. Distinct from output-buffer-full.

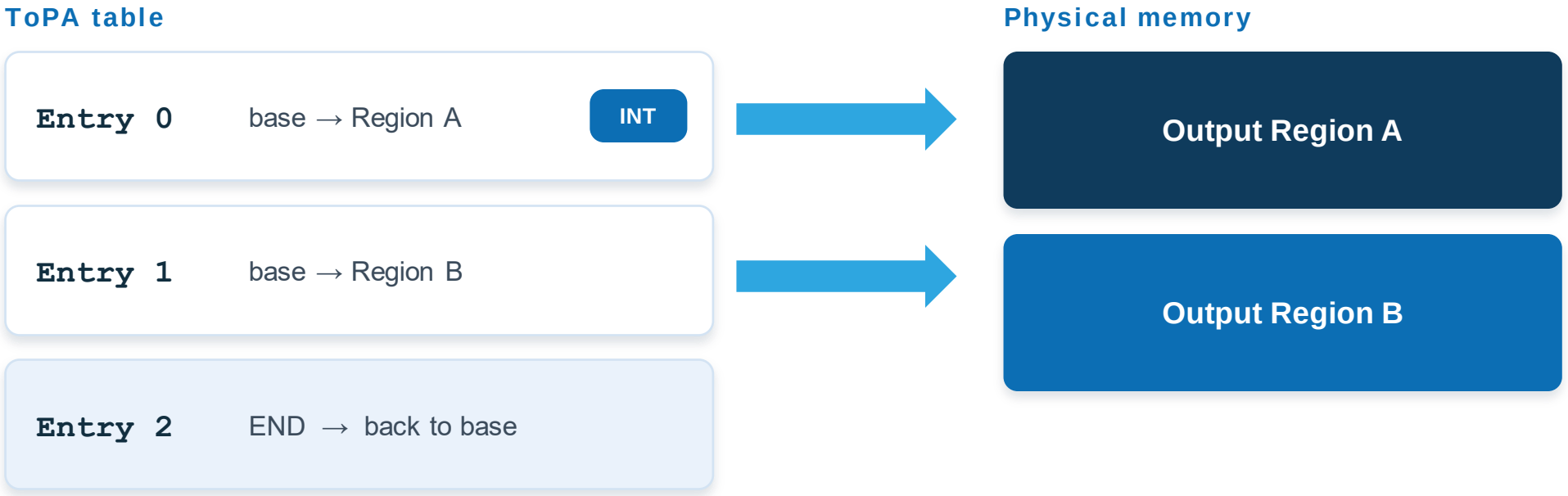
TSC / MTC / CYC
Wall-clock and cycle timing packets for time-accurate reconstruction.

CBR
Core-to-bus ratio , the current frequency, needed to interpret timing.

Two “overflows” to keep apart: OVF = the CPU’s internal FIFO filled (data gap); output-region-full = our RAM buffer filled (triggers a PMI).

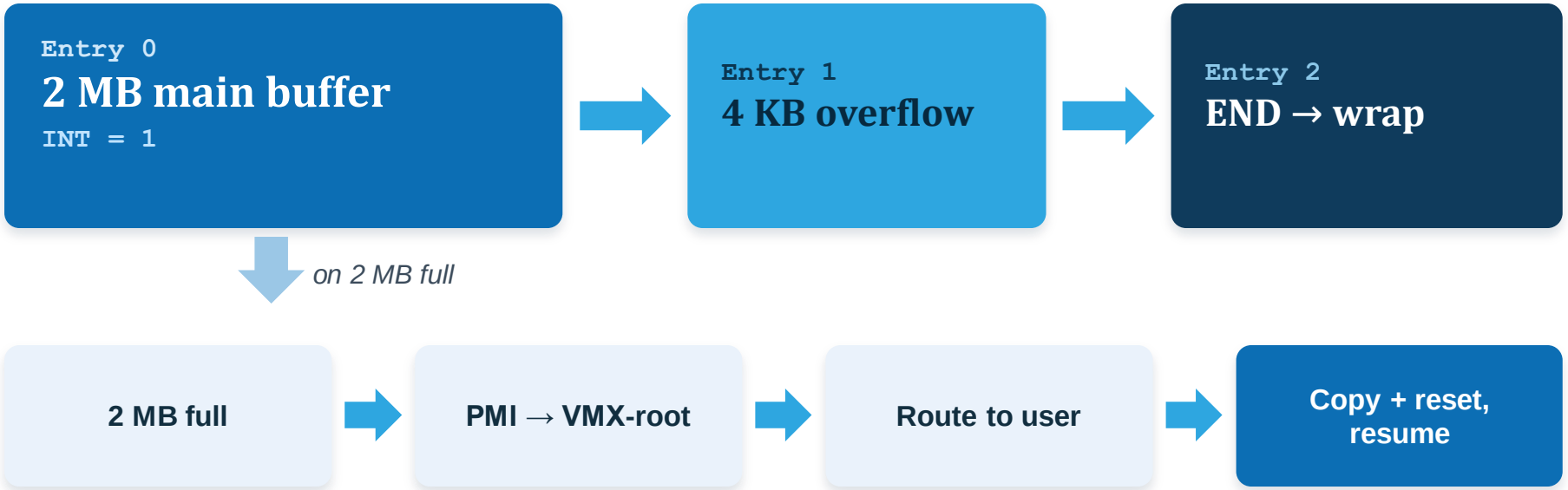
ToPA: Table of Physical Addresses

PT output is directed to memory through a linked list of output regions. Each entry carries a base, a size, and attribute bits.



Region sizes are powers of two, 4 KB – 128 MB. **Hardware tracks position with RTIT_OUTPUT_BASE (table) and RTIT_OUTPUT_MASK_PTRS (current entry + offset).**

Buffer + overflow + PMI hand-off



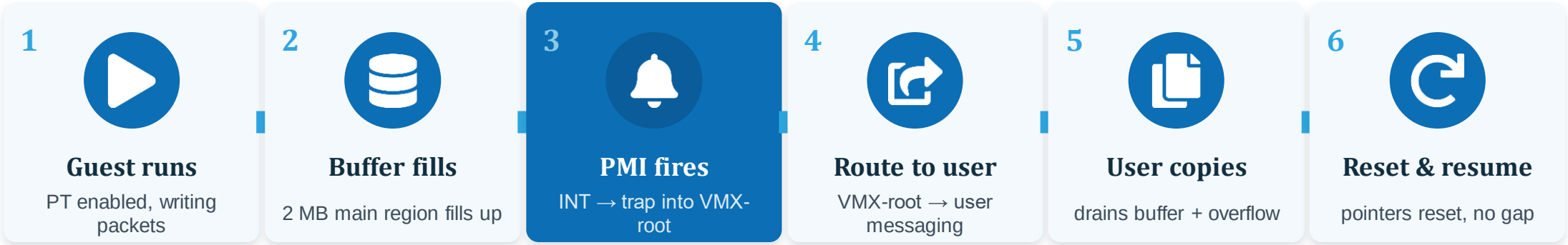
Why a 4 KB overflow?

- The INT bit fires a PMI but tracing does not stop (INT, not STOP).
- PT keeps writing during PMI-delivery latency.
- The 4 KB region absorbs those in-flight packets → zero trace loss.

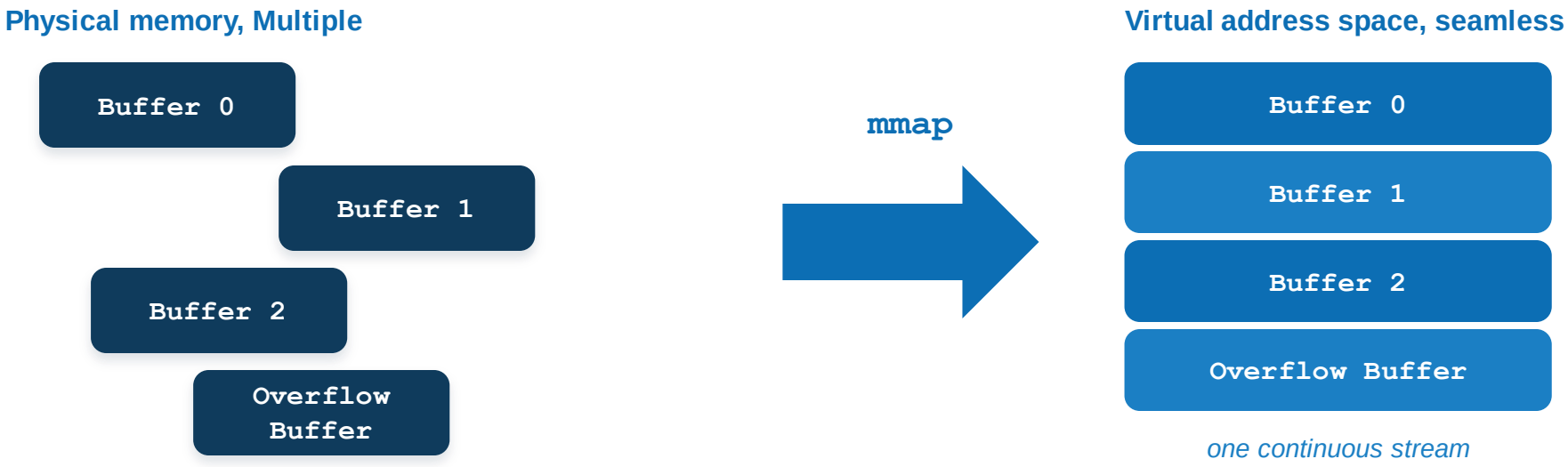
Buffer size is configurable via `pt size`. 2 MB is the default.

Trace-collection workflow

From the guest running to a decoded buffer in the user's hands, continuous and transparent to the target.



Mmap, one contiguous view



Transparent the user reads a single seamless buffer

No copies decode straight from mapped memory

No bookkeeping physical layout stays hidden

The command surface

A small, composable set of commands drives the whole pipeline.

`pt enable`	Start tracing, set TraceEn and arm the ToPA output
`pt filter`	Scope the trace: process (CR3), IP range (ADDR0–ADDR3), or privilege
`pt pause`	Suspend tracing (clear TraceEn) while keeping the configuration
`pt resume`	Re-arm TraceEn and continue the same session
`pt disable`	Stop tracing and tear the session down
`pt size`	Configure the trace-buffer size (the ToPA output region)
`pt flush`	Flushes the buffers
`pt mmap`	Expose the buffer to the user as one contiguous virtual range

Two decoders, pay for detail on demand

Simple / fast decoder

no binary required

- Consumes the packet stream directly.
- Emits branch decisions (TNT) plus indirect jump / call / return target addresses.
- High-throughput and lightweight.
- Ideal for coverage and fast triage.

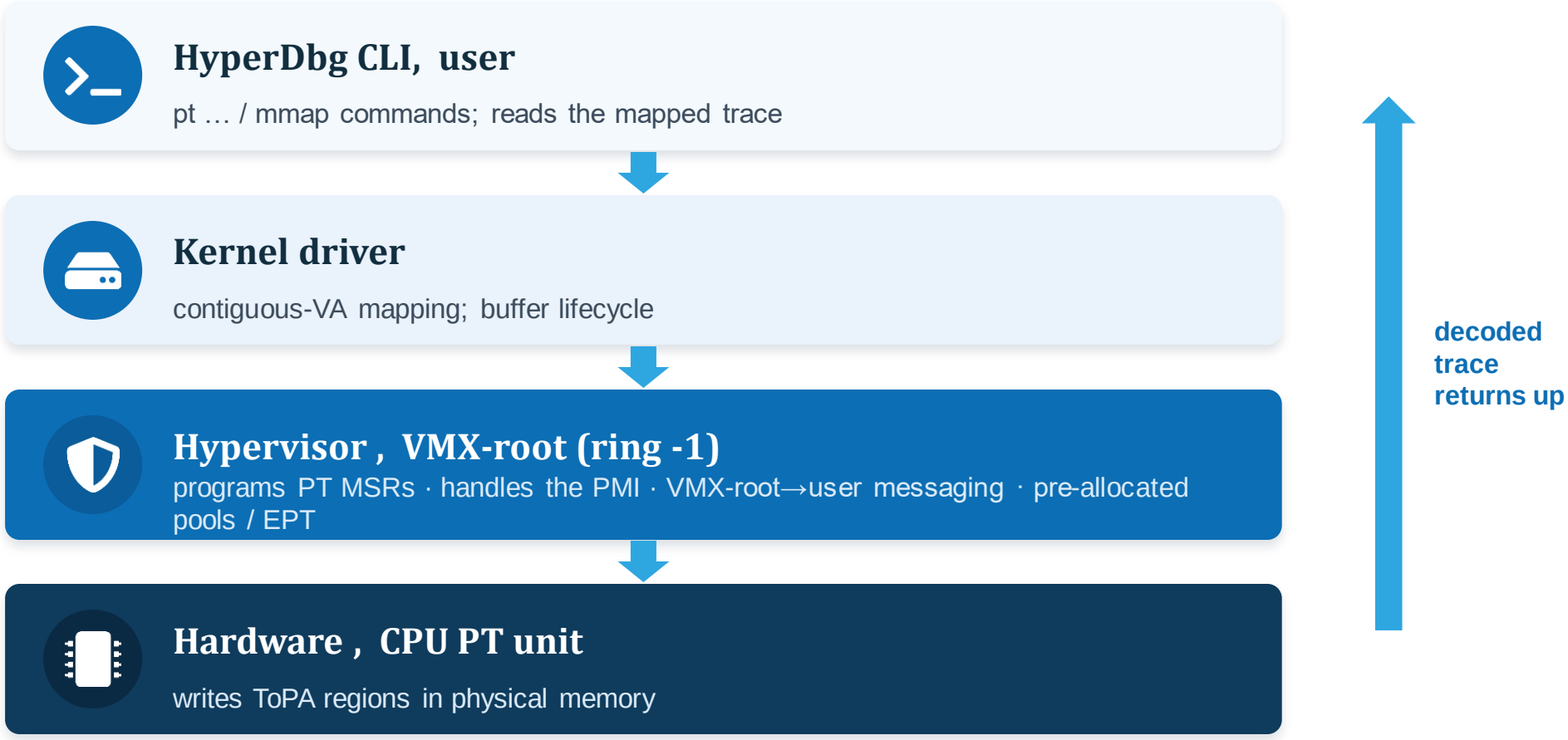
Full / reconstruction decoder

uses the binary's .text section

- Walks the disassembly, guided by TNT / TIP.
- Reconstructs every executed instruction.
- Yields a full instruction trace and basic-block / edge coverage.
- Precise control flow for deep analysis.

Same trace, two fidelity levels, spend decode time only when you need per-instruction detail.

Where PT lives inside HyperDbg



Solved on the way: allocating memory in VMX-root · handling the PMI · staying transparent to the guest.

IMPACT

What this enables

Coverage-guided fuzzing

Turn decoded flow into edge bitmaps (kAFL / Nyx-style) to drive AFL++,the direct link to hypervisor fuzzing.

Reverse engineering & malware

Stealthy, OS-independent instruction traces that resist anti-debugging and time-delta detection.

Post-mortem / reverse debugging

Reconstruct exactly what executed in the moments before a crash, no re-run required.

Control-flow & performance analysis

Hot paths, coverage measurement and anomaly detection from a faithful execution record.

Challenges & where it goes next

Challenges met

- PMI-delivery latency window, solved with the 4 KB overflow region.
- Memory allocation in VMX-root, solved with pre-allocated pools.
- Decode cost at scale, two decoders trade speed for detail.
- Keeping the guest fully transparent throughout.

Future work

- Real-time streaming decode instead of batch copy-out.
- Tighter AFL++ / HyperFuzz integration.

IN SUMMARY

Hardware-speed tracing, inside a transparent debugger

- Intel PT gives near-free, hardware control-flow tracing; HyperDbg gives a stealthy ring -1 host.
- A configurable 2 MB + 4 KB ToPA / PMI buffer captures the full trace with no loss, transparently.
- A clean command set, contiguous-VA mmap, and two decoders (fast + full) make it usable.
- A foundation for coverage-guided fuzzing and stealthy execution analysis.

VU VRIJE UNIVERSITEIT AMSTERDAM

Thank you , questions?